

SERVICIO NACIONAL DE APRENDIZAJE
Centro Industrial de Mantenimiento y Manufactura — CIMM
Regional Boyacá

Guía de Ejercicios Prácticos

Introducción a Servlets Java con MySQL y Patrón DAO

Programa: Tecnología en Análisis y Desarrollo de Software (ADSO)
Nivel: Tecnólogo · Trimestre 4 · Área: Programación Web
Tecnologías: JDK 14 · Apache Tomcat 8.5 · MySQL 8.33+

Presentación

Esta guía contiene un ejercicio práctico para el desarrollo de aplicaciones web con Servlets Java, MySQL y el patrón de diseño DAO (Data Access Object). Este ejercicio está contextualizado en entidades reales del departamento de Boyacá, permitiendo a los aprendices del programa ADSO aplicar los conocimientos en escenarios cercanos a su entorno.

El ejercicio tiene el mismo nivel de complejidad y la misma arquitectura requerida, de modo que el aprendiz que domine uno podrá replicar el patrón en los demás.

Competencias que se desarrollan

Competencia	Resultado de Aprendizaje
Construir aplicaciones web con Servlets Java	Implementa el ciclo GET/POST con HttpServlet
Aplicar el patrón DAO	Separa la lógica de acceso a datos del controlador
Conectar Java a MySQL con JDBC	Gestiona conexiones y ejecuta CRUD con PreparedStatement
Desarrollar vistas con JSP	Usa scriptlets para renderizar datos desde el servlet
Aplicar buenas prácticas	Usa try-with-resources, previene SQL Injection

Restricciones técnicas comunes a todos los ejercicios

Componente	Versión / Requisito
JDK	14
Servidor de aplicaciones	Apache Tomcat 8.5
Base de datos	MySQL 8.0.33+ o MariaDB 10+
Driver JDBC	mysql-connector-java-8.x.x.jar
IDE recomendado	NetBeans 12+ con soporte Java EE
Frameworks	NO se permite Spring, Hibernate ni JSTL
Consultas SQL	Obligatorio usar PreparedStatement (nunca concatenar SQL)
Conexiones JDBC	Obligatorio cerrar con try-with-resources

E1

Registro de Productores Agropecuarios — UMATÁ Ventaquemada

Descripción del caso

La Unidad Municipal de Asistencia Técnica Agropecuaria (UMATA) del municipio de Ventaquemada, Boyacá, lleva el registro de sus productores agrícolas en hojas de cálculo desactualizadas. El alcalde municipal ha solicitado un sistema web para gestionar la información de los productores que reciben asistencia técnica en cultivos de papa, cebolla, zanahoria y arveja, productos emblema del municipio.

Funcionalidades requeridas (CRUD completo)

#	Funcionalidad	Descripción
1	Registrar productor (POST)	Guardar nuevo productor en la base de datos
2	Listar productores (GET)	Mostrar todos en tabla con producto y hectáreas
3	Buscar por cédula o nombre (GET)	Localizar productor específico
4	Editar productor (POST)	Actualizar datos del productor seleccionado
5	Eliminar productor (GET)	Retirar productor del sistema con confirmación

Tabla: productores

Campo	Tipo MySQL	Restricción	Descripción
id	INT	AUTO_INCREMENT, PK	Identificador único
nombres	VARCHAR(80)	NOT NULL	Nombres del productor
apellidos	VARCHAR(80)	NOT NULL	Apellidos del productor
cedula	VARCHAR(20)	NOT NULL, UNIQUE	Número de cédula
vereda	VARCHAR(80)	NOT NULL	Vereda de residencia
telefono	VARCHAR(20)	NOT NULL	Teléfono de contacto
correo	VARCHAR(100)	NULL	Correo electrónico (opcional)
producto_principal	VARCHAR(40)	NOT NULL	papa, cebolla, zanahoria, arveja, otro
hectareas	DECIMAL(6,2)	NOT NULL	Área cultivada en hectáreas
tiene_riego	TINYINT(1)	NOT NULL, DEFAULT 0	1=Sí, 0=No
observaciones	VARCHAR(300)	NULL	Notas adicionales
fecha_registro	DATETIME	DEFAULT NOW()	Fecha de registro automática

Arquitectura requerida (paquete: co.umata.ventaquemada)

```
src/co/umata/ventaquemada/
  model/Productor.java          <- POJO con todos los atributos
  util/ConexionDB.java          <- URL JDBC + obtenerConexion()
  dao/ProductorDAO.java         <- insertar, listar, buscar, actualizar, eliminar
  servlet/ProductorServlet.java <- enrutamiento por parametro 'accion'
  filter/EncodingFilter.java    <- UTF-8
web/WEB-INF/vistas/
  listado.jsp      formulario.jsp  confirmacion.jsp
sql/umata_ventaquemada.sql
```

Criterios de evaluación

Criterio de Evaluación	Puntos
Script SQL correcto (BD + tabla + 5 datos de prueba)	10
ConexionDB.java con parámetros correctos	10
Productor.java con todos los atributos y get/set	10
ProductorDAO — insertar() con PreparedStatement	15
ProductorDAO — listarTodos() con ResultSet	15
ProductorDAO — buscarPorCedula() y buscarPorNombre()	10
ProductorDAO — actualizar() y eliminar()	10
ProductorServlet — enrutamiento GET/POST por acción	10
Vistas JSP funcionales (formulario + listado)	10
Total	100

Pistas

Campo hectareas — usar double en Java y setDouble() en JDBC:

```
// En Productor.java:
private double hectareas;

// En ProductorDAO.java — insertar():
ps.setDouble(9, productor.getHectareas());

// En mapearFila():
p.setHectareas(rs.getDouble("hectareas"));
```

Campo tiene_riego — checkbox, igual que primeraDonacion:

```
// En el Servlet:
boolean tieneRiego = request.getParameter("tieneRiego") != null;
ps.setBoolean(10, tieneRiego);

// En formulario.jsp:
☐

```

Select de veredas de Ventaquemada:

```
<select name="vereda">
    <option value="Centro">Centro</option>
    <option value="Gaunza Arriba">Gaunza Arriba</option>
    <option value="Gaunza Abajo">Gaunza Abajo</option>
    <option value="Paramo de Guerrero">Paramo de Guerrero</option>
    <option value="Puente Piedra">Puente Piedra</option>
    <option value="Recycue">Recycue</option>
    <option value="Otra">Otra</option>
</select>
```

Mostrar hectáreas con formato decimal en el JSP:

```
<% String ha = String.format("%.2f ha", productor.getHectareas()); %>
Hectareas cultivadas: <%= ha %>
```

Especificaciones del Entregable

Estructura mínima del ZIP a entregar

```
NombreAprendiz_EjercicioN/  
  src/          <- paquetes Java completos  
  web/          <- JSPs, CSS, JS, web.xml  
  sql/          <- script .sql con CREATE + INSERT  
  README.txt    <- pasos para ejecutar el proyecto
```

Verificación antes de entregar

Punto a verificar	Cómo comprobarlo
MySQL está corriendo	Ver servicios de Windows o XAMPP
La BD fue creada con el script SQL	Ejecutar <code>SELECT * FROM tabla</code> en Workbench
El driver .jar está en WEB-INF/lib/	Verificar en el explorador de archivos
La contraseña en ConexionDB.java es correcta	Revisar el campo CLAVE
Tomcat 8.5 está configurado en NetBeans	Tools → Servers
El proyecto compila sin errores	Build → Clean and Build
El CRUD completo funciona en el navegador	Probar las 5 operaciones

Investigación

1. **Integre un chatbot a la aplicación web.** Para su implementación puede utilizar cualquiera de las siguientes plataformas gratuitas:
 - **Botpress** (<https://app.botpress.cloud>): plataforma especializada en chatbots con inteligencia artificial. Permite diseñar flujos de conversación visuales y se integra mediante un fragmento `<script>` en el JSP. Cuenta con plan gratuito y documentación en español.
 - **Tawk.to** (<https://www.tawk.to>): Es 100% gratuita sin límite de mensajes, se integra mediante un único fragmento `<script>` en el JSP y su panel de administración está disponible en español.
 - **Crisp** (<https://crisp.chat>): opción de diseño moderno. Cuenta con plan gratuito generoso y una interfaz visual limpia que se integra de forma coherente con Bootstrap.
 - **Dialogflow de Google** (<https://dialogflow.cloud.google.com>): opción de mayor profundidad técnica. Permite conectar el chatbot con la API REST del proyecto y explorar conceptos como intenciones, entidades y webhooks. Requiere cuenta de Google y es gratuita.

El chatbot debe estar visible en todas las páginas de la aplicación y responder preguntas básicas relacionadas con el contexto del sistema.

2. Reemplace el CSS personalizado por el framework **Bootstrap 5** y aplique la validación de formularios nativa de Bootstrap, garantizando que todos los campos obligatorios sean verificados antes del envío al servidor.
3. Implemente un sistema básico de roles de usuario. El rol **administrador** tendrá acceso completo al CRUD, mientras que el rol **consulta** únicamente podrá visualizar el listado de registros sin acceso a las acciones de crear, editar ni eliminar.